

Agro-Smart

IoT-Based Smart Agriculture
Monitoring System

2nd Progress Presentation

- KAHNDSE25.1F-003
- KAHNDSE25.1F-005
- KAHNDSE25.1F-007
- KAHNDSE25.1F-009
- KAHNDSE25.1F-032



Introduction

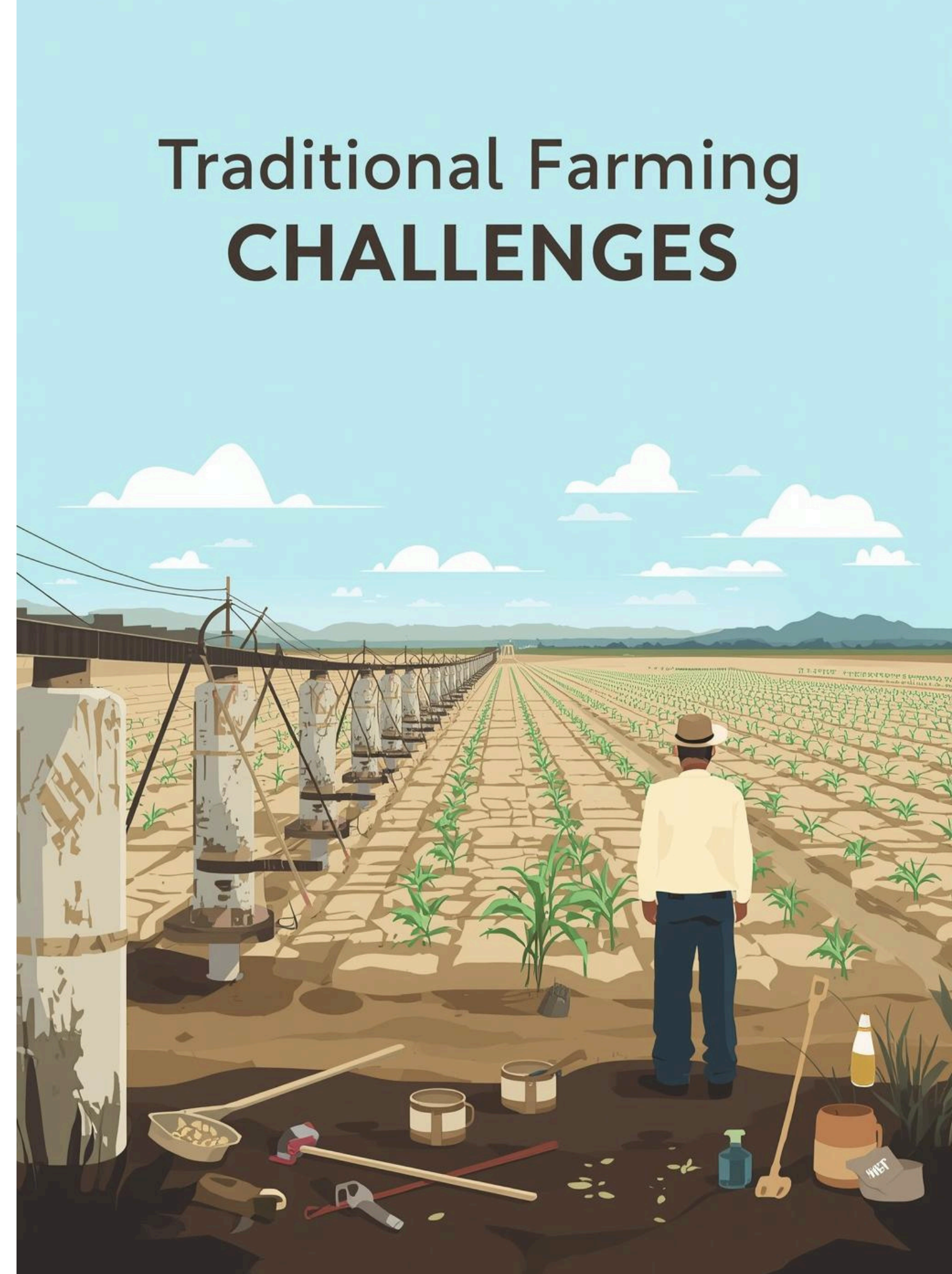
- Agriculture depends heavily on environmental conditions
- Traditional farming lacks real-time monitoring and data-driven decisions
- Water wastage and inefficient crop planning are common issues
- IoT, data analytics, and AI can modernize agriculture
- This project proposes a complete smart agriculture solution



Problem Statement

- Lack of real-time monitoring in traditional farming.
- Inefficient water usage due to manual irrigation.
- Difficulty in analyzing environmental trends.
- Farmers do not have easy access to data-driven insights.
- Limited use of technology in small-scale farming.

Traditional Farming **CHALLENGES**



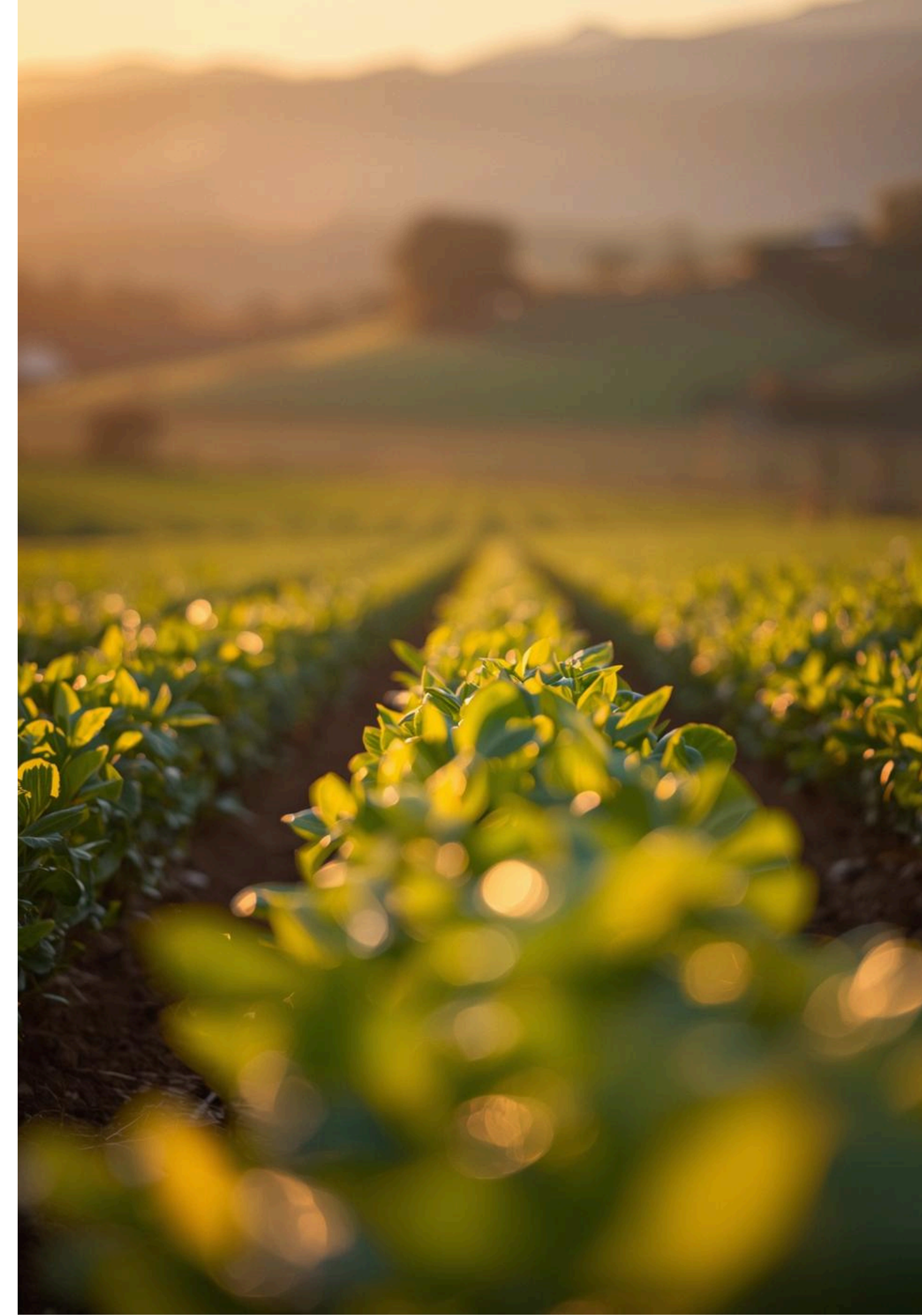
Aim & Objectives

Aim of the Project

- To develop an IoT-based smart agriculture monitoring system that collects real-time environmental data and uses data analysis and AI/ML techniques to provide actionable insights for improving irrigation efficiency and crop suitability.

Objectives of the Project

- To design and implement an IoT-based agriculture monitoring system using sensors and ESP32.
- To collect real-time data such as temperature, humidity, soil moisture, and light intensity.
- To store and visualize sensor data using a dashboard and a mobile application
- To analyze historical datasets and live sensor data to identify patterns and trends.
- To provide smart irrigation recommendations and crop-related insights based on analyzed data using ML and AI.

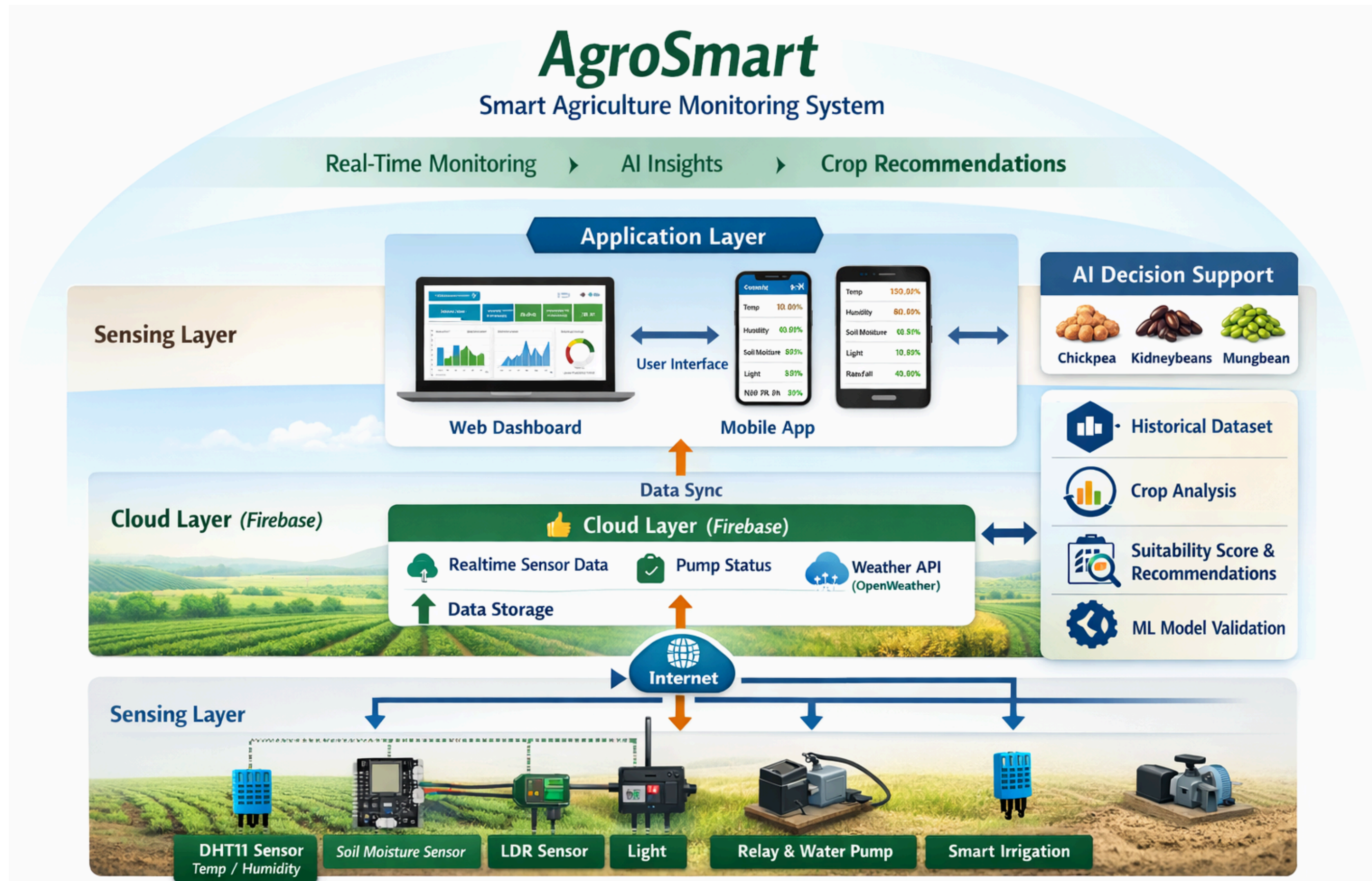


System Overview

- IoT sensors collect environmental data
- ESP32 sends collected data to Firebase
- N, P, K, and pH values will be taken via a mobile application or a web application
- Weather data will be collected from the API (Openweathermap) and sent to Firebase
- Firebase sends Data to the dashboard and mobile application
- The AI and ML module provides decision support through analysis (Two layers)
- Dashboards display live data, analytics, and recommendations



Architecture





Methodology Overview

Methodology Overview

System Design & Data Collection

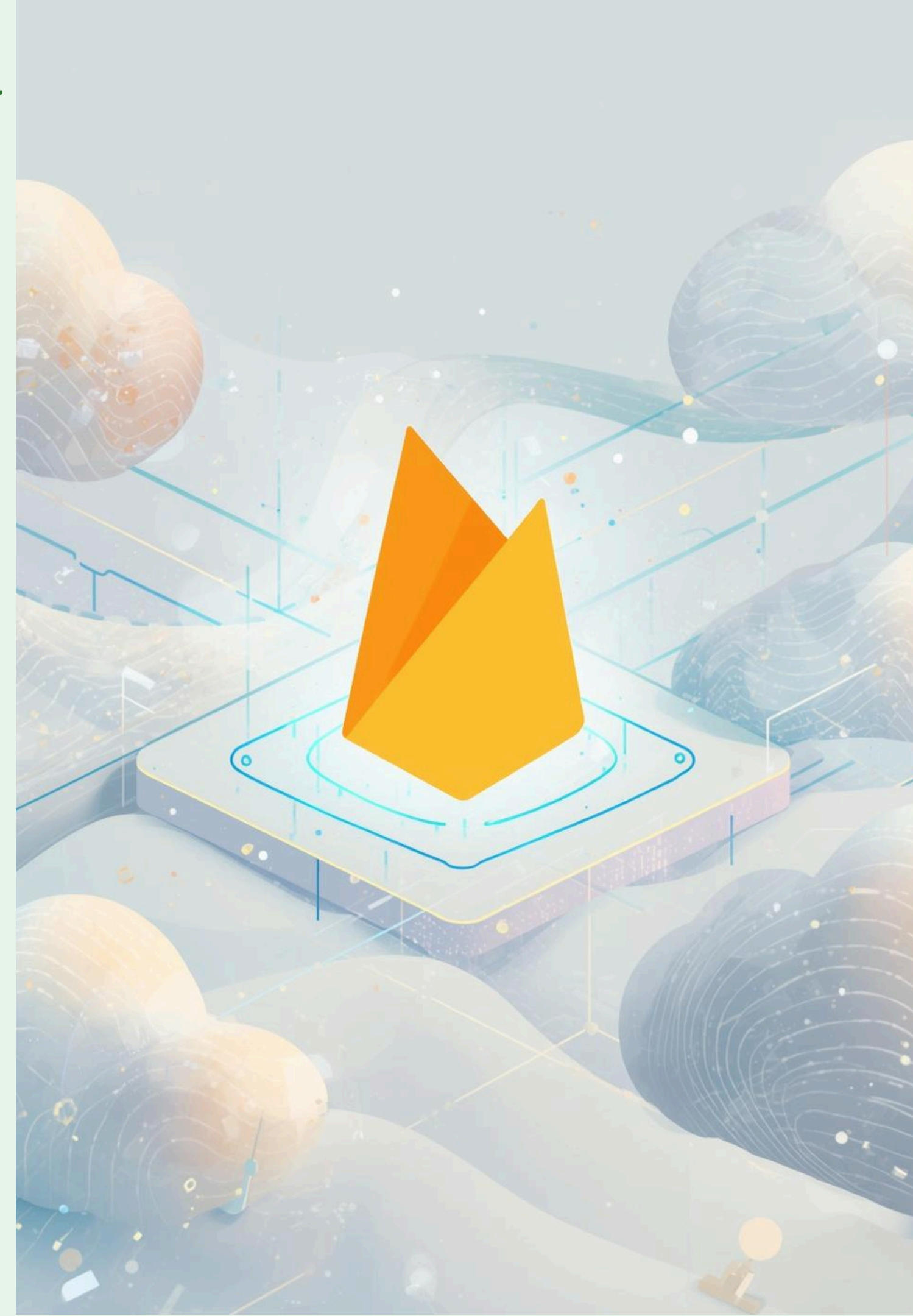
- Adopted a monitoring-first architecture
- Designed a modular IoT-based system
- Environmental data collected using:
 - ESP32 microcontroller
 - Temperature & humidity sensor (DHT11)
 - Soil moisture sensor
 - Light intensity sensor (LDR)
- Smart irrigation implemented using:
 - Soil moisture threshold
 - Relay-controlled water pump
- Real-time sensor data transmitted to cloud via Wi-Fi



Methodology Overview

Cloud, Dashboard & Analytics

- Firebase Realtime Database used for:
 - Sensor data storage
 - Pump status monitoring
 - AI insights storage
- Web dashboard developed using:
 - React + Vite
 - Tailwind CSS
- Dashboard functions:
 - Real-time data visualization
 - Irrigation status monitoring
 - Rainfall integration via OpenWeatherMap API
- Analytics approach:
 - Historical crop patterns simulated from dataset
 - Live sensor data appended in real time
 - Unified timeline: historical → live



Methodology Overview

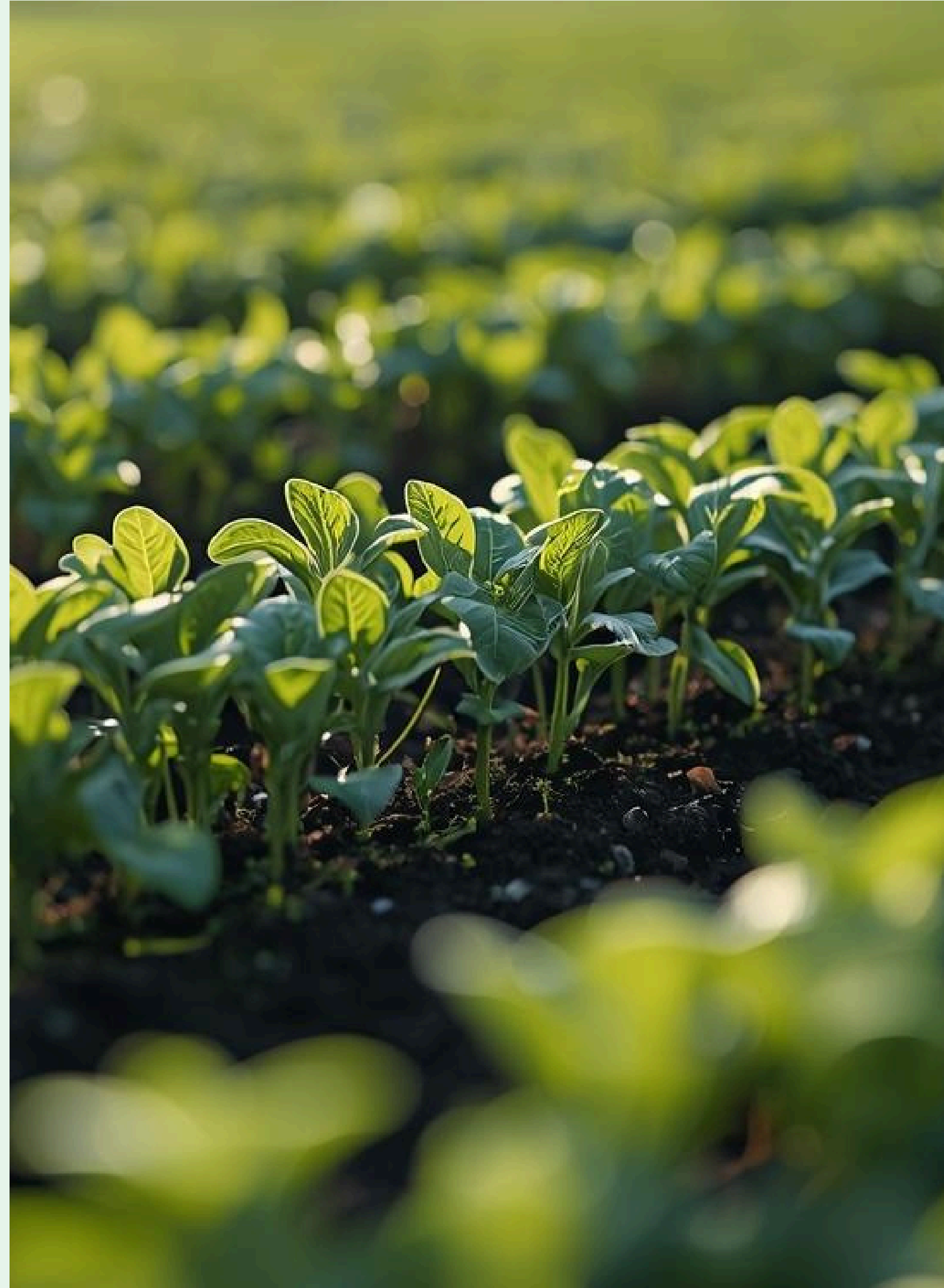
AI Decision Support & Validation

- Historical dataset analyzed (Kaggle Crop Recommendation Dataset)
- Scope restricted to three crops only:
 - Chickpea
 - Kidneybeans
 - Mungbean
- AI method:
 - Normalized Manhattan-distance similarity
 - Real-time data compared with historical crop patterns
- AI outputs:
 - Crop suitability score
 - Match level (Good / Moderate / Poor)
 - Explainable reasons ("Why this crop?")
- ML extension:
 - Random Forest model trained separately
 - Used as secondary validation only



Current Progress

- Hardware setup completed
- Sensors integrated and tested
- N, P, K, Values, pH value, weather data collection feature implemented
- Firebase real-time data working
- Dashboard developed and stable
- AI crop recommendation implemented
- ML Model trained
- Rainfall API integrated
- Mobile application in progress(70% completed)



Challenges Faced

Key difficulties in project development



Technical Challenges

- Firebase Cloud Functions required payment (budget limitation)
- Kaggle dataset not in time-series format
- Feature scale imbalance affecting AI results
- Environment variable issues during deployment
- Problems in collecting weather data
- Collecting N, P, K, values (Budget limitations)
- Training a ML model

Solutions Implemented

- Client-side AI and analytics (no Cloud Functions)
- Simulated historical baseline for analytics
- Min-Max normalization for AI fairness
- Stable Netlify configuration for hosting
- Used API from openweathermap.org
- Added a manual method to enter values for N, P, K, and pH in the hope of automation in the future
- Used

Conclusion

- Hardware setup completed.
- Sensors tested and calibrated.
- ESP32 successfully connected to WiFi.
- Real-time data uploaded to Firebase.
- Smart irrigation logic implemented.
- AI recommendation model implemented (1st layer)
- Trained ML model implemented (2nd Layer)
- Web dashboard developed and hosted



Thank you!



AgroSmart™

